Pwnagotchi — Complete Hardware Guide

> **Image:** Pwnagotchi (Raspberry Pi OS-based `.img`) · **Upstream:** jayofelony/pwnagotchi (GPL-3.0) — the actively maintained fork **Boards:** Raspberry Pi Zero 2 W / Zero W (also Pi 3/4/5) · **Cyber Controller profile:** `pwnagotchi` (`sd` backend — removable-only SD/USB image writer, **not** esptool/serial flashing) **This guide:** purchase → build → write SD image via Cyber Controller → boot/configure → use → troubleshoot.

1. Overview

Pwnagotchi is a Raspberry Pi WiFi appliance that passively roams its environment and harvests **crackable WPA/WPA2 key material** — full and half 4-way handshakes and PMKIDs — saving them as `.pcap` files that feed straight into hashcat. Under the hood it drives **bettercap**, capturing passively where it can and performing association/deauthentication attacks where it must to elicit a handshake. It presents as a Tamagotchi-style pet with animated faces/moods on a small e-ink screen.

About the "AI": the original project (evilsocket) used deep reinforcement learning (an A2C actor-advantage-critic agent) to tune its own scanning parameters and "learn" its environment. The maintained **jayofelony fork removed the onboard AI** — its README states the AI "seemed to destabilize the Wi-Fi firmware," so it was dropped to improve uptime and battery life. Capture is now driven deterministically by bettercap while keeping the pet aesthetic. (Treat any "reinforcement learning" framing as the project's heritage, not a feature of current images — verify against the release you flash.)

Cyber Controller's role here is **imaging**, not serial control: it downloads the latest Pwnagotchi `.img` release, decompresses it, and writes it block-level to a removable microSD. Once booted, you interact with the Pi over its USB gadget link (SSH + web UI), not over a firmware serial protocol.

2. Legal & Safety

Authorized testing only. Pwnagotchi does not merely listen — to force handshakes it sends **deauthentication and association frames** on the 2.4 GHz band, which is **illegal against networks and people you do not own or have written permission to test** in most jurisdictions (US: CFAA/FCC; EU and others similar). Run it only on your own lab APs or under an explicit, scoped engagement. - **Capture and cracking are separate, separately-justified acts.** Capturing a handshake is one thing; **cracking** it is an offline step you perform later on **your own captures** — never crack material you were not authorized to collect. - Passive sniffing alone may still be regulated where you live — check local law. - A roaming, always-on collector is easy to forget about; treat the captures it accumulates as sensitive and purge them when done.

3. Hardware & Purchasing

Pwnagotchi targets the small, low-power Raspberry Pi boards. Pick by portability vs. horsepower:

Tier	Board	Why	Where to buy (search terms)
Best pocket build	**Raspberry Pi Zero 2 W** (64-bit, quad-core)	Recommended — small, low-draw, enough CPU; the fork's 64-bit target	Raspberry Pi Approved Resellers; search "Raspberry Pi Zero 2 W" (Adafruit, PiShop, CanaKit, Pimoroni)
Lowest power	**Raspberry Pi Zero W** (32-bit, single-core)	Tiniest/cheapest; slower, 32-bit image	Approved resellers; search "Raspberry Pi Zero W"
Bench / faster	**Raspberry Pi 3 / 4 / 5** (64-bit)	More CPU/RAM, USB host; less pocketable	Approved resellers; search "Raspberry Pi 4 Model B"

Display (optional but typical) — e-ink keeps the pet face visible with no power draw:

Display	Notes	Search terms
Waveshare 2.13" e-Paper HAT (V2/V3/V4)	The classic Pwnagotchi screen; 250×122 mono. Match your panel revision to the config string	"Waveshare 2.13 e-Paper HAT" (note the V2/V3/V4 revision on the box)
Waveshare 1.54" / 2.9" e-Paper	Alternative e-ink sizes the fork supports	"Waveshare 1.54 e-Paper", "Waveshare 2.9 e-Paper"
OLED / small LCD (e.g. Display HAT Mini, OLED HAT)	Color/refresh but draws power continuously	"Pimoroni Display HAT Mini", "I2C OLED 128x64 HAT"
None (headless)	Use the web UI / SSH only — set `dummydisplay`	n/a

Accessories: - **microSD card:** 16 GB+ recommended, **Class 10 / A1** (8 GB minimum); a quality card and reader matter — flaky cards are the #1 cause of corruption. - **Power / battery:** a clean **5V/2.5A** supply, or for roaming a **USB power bank** or a **PiSugar 2/3** / UPS battery HAT (the project notes a PiSugar partnership — verify current model/capacity). - **USB data cable** (not charge-only) for the gadget link to your PC. - **40-pin GPIO header** on the Pi Zero/Zero 2 W if you'll mount a HAT — the Zero ships **without** a soldered header, so buy a pre-soldered board or solder one on.

Avoid: charge-only cables (no gadget link), no-name microSD cards, and assuming a display revision — a Waveshare 2.13" V2 vs V3 vs V4 mismatch leaves the screen blank.

4. Building / Assembly

- **Headless (no screen):** nothing to assemble — just a microSD and a data cable. Set `ui.display.enabled = false` (or type `dummydisplay`) and use the web UI/SSH.
- **With an e-ink/OLED HAT:** seat the HAT on the Pi's **40-pin GPIO header** (header must be present — solder one onto a bare Zero/Zero 2 W first). The Waveshare 2.13" HAT presses straight onto all 40 pins. Press firmly and evenly; a partially seated HAT shows nothing.
- **Identify your panel revision now.** The Waveshare 2.13" ships as **V1/V2/V3/V4**, and the revision determines the config string (V2→`waveshare_2`, V3→`waveshare_3`, V4→`waveshare_4`). Check the sticker

on the back of the panel before you close the case.

- **Know your USB ports (Pi Zero / Zero 2 W):** there are two micro-USB jacks. The **inner one labeled "USB"** is the data/OTG port used for the gadget link and SSH; the **outer "PWR IN"** is power-only. Plug your PC into the **"USB"** port.
- **Battery/case (portable builds):** mount the PiSugar/UPS HAT or attach the power bank, then a case. e-ink retains its last image with the board off, so the face persists when powered down.

5. Write the SD image (via Cyber Controller)

Pwnagotchi is **not flashed over serial** — Cyber Controller writes a whole-disk `.img` to a **removable** microSD. The writer refuses fixed/system disks, refuses drives ≥ 256 GB, requires an explicit removable target plus confirmation, and needs **Administrator/root** for raw disk access.

1. Insert the microSD into a card reader on your PC. **Everything on it will be erased.**
2. Open Cyber Controller's **SD / Software-OS imaging** flow (the removable-image writer; firmware boards live on the serial Flash tab — Pwnagotchi is not there).
3. **Image profile:** `Pwnagotchi (jayofelony)` — Cyber Controller queries the latest GitHub release (`jayofelony/pwnagotchi`) and lists matching `pwnagotchi-*.img.(xz|gz|zip)` assets.
4. **Target drive:** click **Refresh drives** and pick your card — **only removable, SD-sized drives are listed** (the safety guard hides system disks). Confirm the size/name matches your card.
5. **Write** and confirm the destructive-erase prompt. Cyber Controller then, in one pipeline: downloads the release asset → streams-decompresses it (xz/gz/zip, never loading the full image into RAM) → writes block-level (ctypes `PhysicalDrive` on Windows; `dd` on Linux/macOS) → **verifies by reading back and comparing SHA-256**.
6. On success, eject the card. The image is unconfigured at this point — first boot config comes next.

> If Cyber Controller can't reach GitHub, download a `pwnagotchi-*.img.xz` from the upstream releases page yourself and point the writer at the local image (it still enforces the removable + confirmed + verify guards).

6. Integrate into Cyber Controller

- **Profile:** `pwnagotchi` — `backend: "sd"`, `protocol: null`. The id maps to the SD backend's `PI_IMAGE_PROFILES["pwnagotchi"]` (repo `jayofelony/pwnagotchi`, asset pattern `pwnagotchi.*\.img\.(xz|gz|zip)$`). Because there is no firmware serial protocol, **Cyber Controller's job is imaging only** — there is no Devices-tab live command set for this profile the way there is for serial firmwares like Marauder.
- **First-boot config (`config.toml`):** the device reads `/etc/pwnagotchi/config.toml`. The fastest path is to drop a `config.toml` onto the **boot partition** before first boot (or SSH in and edit `/etc/pwnagotchi/config.toml`, or run the on-device wizard). Keys you'll set:
- `main.name = "your-pet-name"`
- `main.whitelist = ["YOUR_HOME_SSID", "aa:bb:cc:dd:ee:ff"]` — **add your own networks here so it never attacks them.**
- `ui.display.enabled = true` and `ui.display.type = "waveshare_2" | "waveshare_3" | "waveshare_4"` (match your panel; headless → `false` or `dummydisplay`). Color panels may use `ui.display.color`.

- `ui.web.username` / `ui.web.password` — **change from the default `changeme` / `changeme`.**
- Capture behavior lives under `personality.*` (e.g. `personality.advertise`, `personality.deauth`, `personality.associate`, `personality.channels`, `personality.recon_time`) — keep `deauth` enabled only for authorized testing.
- **Connect after boot:** the Pi comes up as a **USB gadget (RNDIS/ECM)** on `10.0.0.2`.
- **SSH:** `ssh pi@10.0.0.2` — default login `pi` / `raspberry` (Raspberry Pi OS base). **Change it immediately with `passwd`.**
- **Web UI:** `http://10.0.0.2:8080` (login `changeme` / `changeme` until you change it). On the PC side, make sure the "Raspberry Pi USB gadget" network interface is up (and ordered below Wi-Fi).
- **Captures land in** `/home/pi/handshakes/` (or `/root/handshakes/` — verify the path on your build) as `.pcap` files; pull them off with `scp` for offline cracking.

7. Usage (end-to-end)

1. **Boot:** insert the configured card, power the Pi from a battery/supply. The face appears on the e-ink screen; it starts in **AUTO mode** — autonomously hopping channels, associating, deauthing (where allowed), and capturing.
2. **AUTO vs MANU: AUTO** is the hands-off hunting loop. **MANU (manual)** is a non-attacking maintenance mode used to connect the Pi to a known Wi-Fi (e.g. for internet to upload captures) or to work on it without it transmitting. The current mode is shown on the display. (Verify the exact AUTO↔MANU toggle for your release in the wiki — historically it is a reboot/flag, not a live key.)
3. **Watch progress:** the web UI / screen show captured-handshake counts and nearby APs.
4. **Collect:** copy captures off — `scp -r pi@10.0.0.2:/home/pi/handshakes ./` (verify the path).
5. **Crack offline (your captures only):** convert with `hcxpcapngtool` to hashcat 22000 format, then `hashcat -m 22000 captures.22000 wordlist.txt`. Plugins like **wpa-sec** can auto-upload to an online cracking service, and **bt-tether** can tether the Pi to a phone for internet — enable only what you're authorized to use (verify plugin availability in your image).
6. **Shut down cleanly** before pulling power to protect the SD card (e-ink keeps the last face shown).

8. Troubleshooting

- **Blank screen after boot:** wrong `ui.display.type` for your panel revision (V2/V3/V4 mismatch) or `ui.display.enabled = false`, or a HAT that isn't fully seated on the 40-pin header. Set the matching string and reseat.
- **No `10.0.0.2` / can't SSH:** you're on the **"PWR IN"** port, not **"USB"**; charge-only cable; or the host's "Raspberry Pi USB gadget" NIC is down or ordered above Wi-Fi. Fix the cable/port, bring the interface up, and retry `ssh pi@10.0.0.2`.
- **Writer shows no target / refuses the drive:** the removable-only guard hides fixed/system disks and anything ≥ 256 GB — use an actual SD card in a reader, click **Refresh drives**, and run Cyber Controller as **Administrator/root**.
- **SHA-256 verify MISMATCH:** bad card or reader, or it was unplugged mid-write — re-seat and re-write; replace the card if it repeats.
- **Boot-loop / corruption / won't come up:** flaky power (use 5V/2.5A) or a failing SD card; re-image to a fresh Class-10/A1 card and always shut down cleanly.

- **No handshakes accumulating:** nothing authorized in range, all nearby SSIDs whitelisted, or `personality.deauth/associate` disabled — review `config.toml` (and remember capture is environment-dependent).
- **Locked out / forgot web password:** re-edit `config.toml` over SSH (`ui.web.*`) or re-run the config wizard; reset SSH with `passwd`.

9. Sources

- Upstream (maintained fork): https://github.com/jayofelony/pwnagotchi — README (supported Pi models, GPL-3.0, AI-removed note, latest release), and the **wiki** (Installation / Connecting / Configuration / Customization).
- Project site: https://pwnagotchi.org (getting-started: installation, configuration, plugins) and the community Discord / r/pwnagotchi.
- Cyber Controller profile: `src/config/profiles/pwnagotchi.json` (`backend: "sd"`, repo/latest URLs) and the SD writer `src/core/backends/sd_backend.py` (`PI_IMAGE_PROFILES["pwnagotchi"]`, removable + confirmed + 256 GB guards, streaming decompress, SHA-256 verify).
- Hardware: Raspberry Pi Approved Resellers (Pi Zero 2 W / Zero W / Pi 4); Waveshare 2.13" e-Paper HAT (confirm V2/V3/V4 revision); PiSugar/UPS battery HATs.
- **Verify at build time:** exact `ui.display.type` string for your panel, the current handshake output path, AUTO↔MANU toggle, default credentials, and plugin availability against the specific release you flash — these can change between versions.